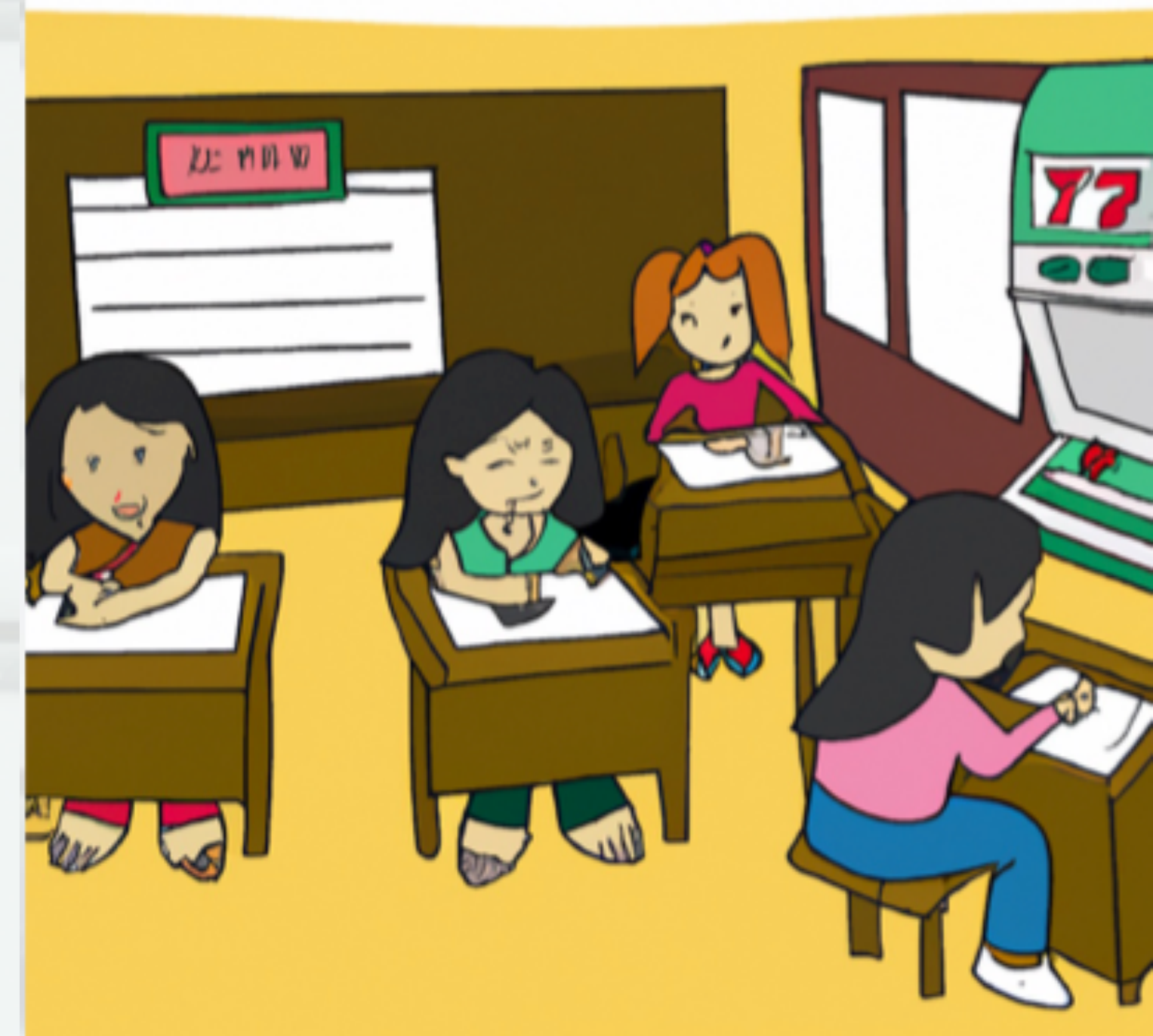


Preparing for the Java 25 cert + Learning New Features

Jeanne Boyarsky
3/17/26
JavaOne



At end of session

<https://speakerdeck.com/boyarsky>

<https://linktr.ee/jeanneboyarsky>

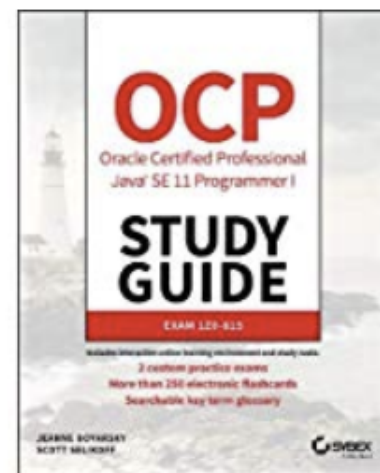
Pause for a Commercial

Amazon Best Sellers

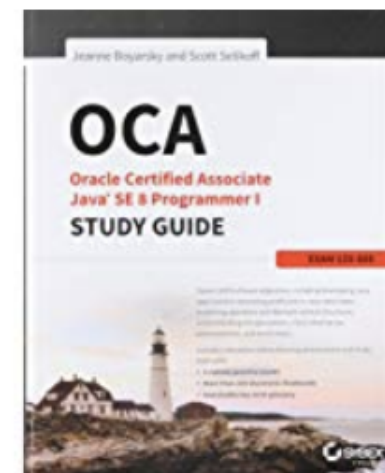
Our most popular products based on sales. Updated hourly.

Best Sellers in Oracle Certification

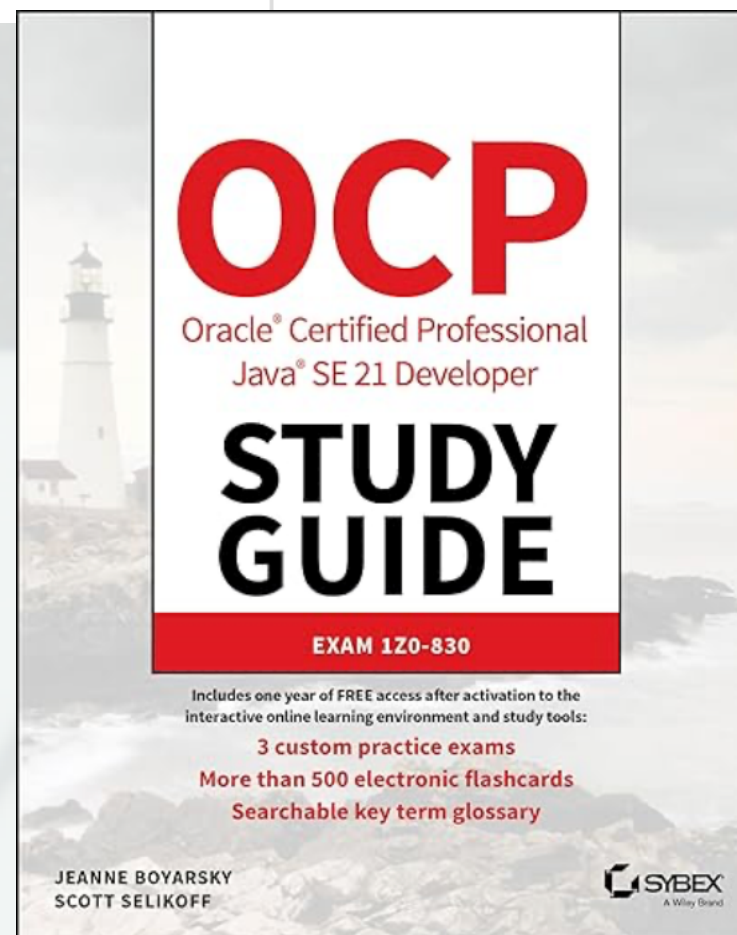
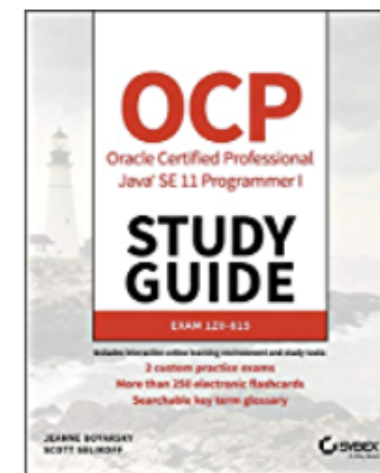
#1



#2

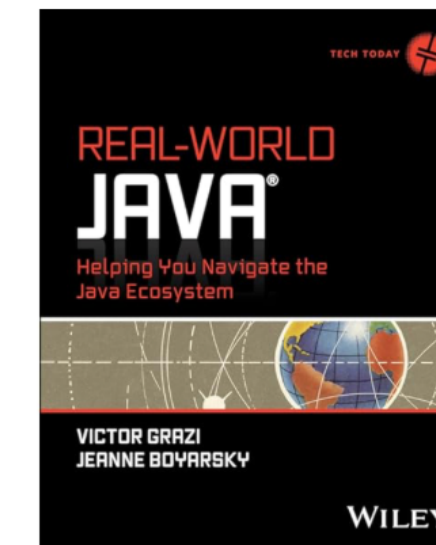


#3



New Releases in Java Programming

#1



Real-World Java: Helping You Navigate the...
> Jeanne Boyarsky
★★★★★ 13
Paperback

#2



Groovy Programming Language for Beginner...
Davis Simon
Paperback
\$19.97

Java certs: 8/11/17/21 (future 25)
Real World Java

<https://linktr.ee/jeanneboyarsky>

Disclaimers

Some of the material in this presentation appears in our certification books.

Agenda - Examples/Questions

- Compact source files (25)
- Instance main (25)
- IO class (25)
- Flexible constructor bodies (25)
- Module imports (25)
- Bonus: JavaDoc (23)
- Unnamed variables/patterns (22)
- Multi file source code (22)

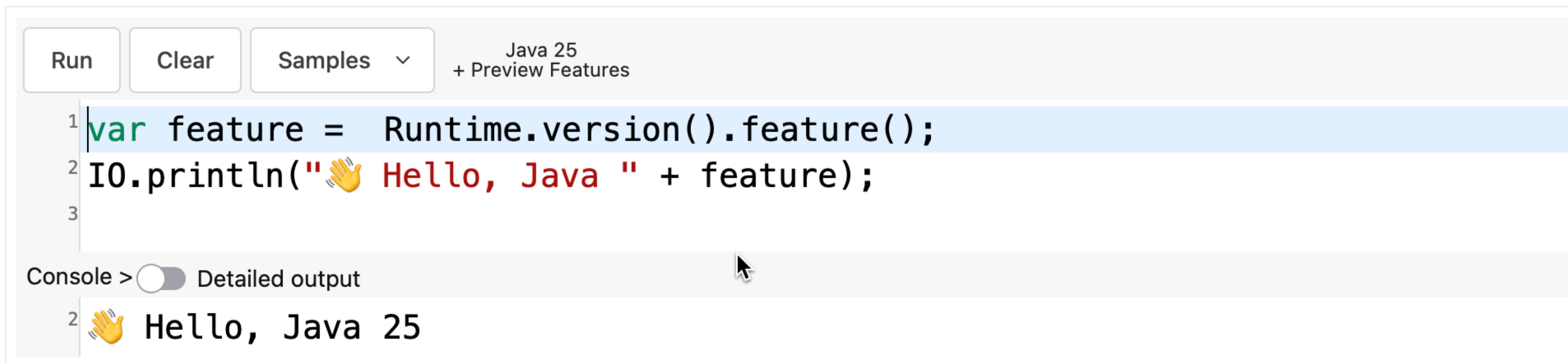
Agenda - Examples/Questions

- Sealed classes (17)
- Instance of/switch patterns (14-21)
- Virtual Threads (21)

Then code!

Locally Java 25+
or <https://dev.java/playground/>
(a little under half can be done in the playground)

The Java Playground



The screenshot shows the Java Playground interface. At the top, there are buttons for 'Run', 'Clear', and 'Samples' with a dropdown arrow. To the right of these buttons, it says 'Java 25 + Preview Features'. Below the buttons is a code editor with three lines of code:
1 `var feature = Runtime.version().feature();`
2 `IO.println("👋 Hello, Java " + feature);`
3
Below the code editor is a console area. It has a label 'Console >' followed by a toggle switch for 'Detailed output'. The console shows the output of the code:
2 👋 Hello, Java 25

Java 22-25 Features

<https://linktr.ee/jeanneboyarsky>

Hello JavaOne

25

```
public class Hello {  
    public static void main(  
        String[] args) {  
        System.out.println(  
            "Hello JavaOne!");  
    }  
}
```

- Compact class files
- Instance main
- IO class

Can use any of these

25

```
void main() {}  
void main(String[] args) {}  
void main(String... args) {}  
static void main() {}  
public void main() {}  
public void main(String[] args) {}
```

+ more

If two, calls one with args

Flexible Constructor Bodies

25

```
public class Sloth {  
    private int age;  
    public Sloth(int ageEntered) {  
  
        Prologue { if (ageEntered < 0) throw new IllegalArgumentException();  
  
        Constructor Call { super();  
  
        Epilogue { this.age = ageEntered;  
  
    }  
}
```


Prologue Rules

25

- Can reference constructor params
- Cannot reference instance methods
- Cannot reference instance variables
 - Except for setting initial value of final one

Constructor Flow - What prints?

25

```
public class Bridge {  
    { IO.print("i"); }  
  
    public Bridge() {  
        IO.print("p");  
        this(1);  
        IO.print("e");  
    }  
  
    public Bridge(int num) {  
        IO.print("n");  
    }  
  
    public void main() {  
        new Bridge();  
    }  
}
```

pinepine

Unnamed Variables

22

- `_` (single underscore) is valid variable name
- Restrictions
 - Cannot read
 - Cannot write
 - Can define multiple in same scope
- Useful for: catch, lambda, pattern matching, etc

Unnamed Variables

22

```
// good  
String _ = "init";  
String _ = "again";
```

```
// no good  
_ = "write";  
IO.println(_);  
String _;
```


Scoped Values

25

```
static final ScopedValue<List<String>>  
    SESSIONS = ScopedValue.newInstance();  
  
void main() {  
    var sessions = List.of("Java 25", "AI");  
    ScopedValue.where(SESSIONS, sessions).run(  
        () -> { attend(); });  
}  
  
void attend() {  
    IO.println(SESSIONS.get());  
}
```

immutable
scoped data

Structured Concurrency

?

Incubated Java 19
Sixth preview in Java 27

Module Imports

25

```
// imports over 50 packages  
import module java.base;
```

```
import java.util.List;  
    VS  
import java.util.*;  
    VS  
import module java.base;
```


Module Imports Shadowing

25

Code	Type	Imports
<code>import module.java.sql;</code>	Module imports	2 direct packages 3 transitive packages (via requires transitive) 1 service (Driver)
<code>import java.sql.*;</code>	Wildcard/package import	50+ classes
<code>import java.sql.Date;</code>	Import single class	1 class

23

@jeanneboyarsky

Bonus: Multi-File Source-Code

22

```
java Conference.java
```

```
public class Conference {  
    public Conference(List<Speaker> speakers) {  
  
    }  
}
```


Module 1 - Question 1

Which lines have compiler errors?

```
1: void main(String[] _) {  
2:     double _ = 0.00;  
3:     String _ = null;  
4:     IO.println(_);  
5:     boolean isNull = _ == null;  
6: }
```

- A. Line 1
- B. Line 2
- C. Line 3

- D. Line 4
- E. Line 5

Module 1 - Question 1

Which lines have compiler errors?

```
1: void main(String[] _) {  
2:     double _ = 0  
3:     String _ = nu  
4:     IO.println(_)  
5:     boolean isNul  
6: }
```

A, D, E

- A. Line 1
- B. Line 2
- C. Line 3

- D. Line 4
- E. Line 5

Module 1 - Question 2

Which types are immutable in SV?

```
static final ScopedValue<XXX>>  
SV = ScopedValue.newInstance();
```

A. List

B. List<String>

C. LocalDateTime

D. String

Module 1 - Question 2

Which types are immutable in SV?

```
static final ScopedValue<XXX>  
SV = ScopedValue.newInstance();
```

C, D

A. List

B. List<String>

C. LocalDateTime

D. String

Module 1 - Question 3

What are true of line 5?

```
1: import java.util.*;  
2: import jeanne.List;  
3: import module java.base;  
4:  
5: void main(List list) {}
```

- A. The import on line 1 is used for line 5
- B. The import on line 2 is used for line 5
- C. The import on line 3 is used for line 5
- D. This program will run

Module 1 - Question 3

What are true of line 5?

```
1: import java.util.*;  
2: import jeanne.  
3: import module  
4:  
5: void main(List
```

B

- A. The import on line 1 is used for line 5
- B. The import on line 2 is used for line 5
- C. The import on line 3 is used for line 5
- D. This program will run

Module 1 - Question 4

What is the output?

```
private class Blue {  
    { IO.print("i"); }  
  
    private Blue() {  
        IO.print("c");  
        this(1);  
    }  
}
```

```
Blue(int num) {  
    IO.print(num);  
    super();  
}  
  
void main() {  
    new Blue();  
}  
}
```

A. cli
B. clicli

C. Does not compile
D. Does not run

Module 1 - Question 4

What is the output?

```
private class Blue {  
    { IO.print("i"); }  
  
    private Blue() {  
        IO.print("c");  
        this(1);  
    }  
}  
  
Blue(int num) {  
    IO.print(num);  
    super();  
}  
  
void main() {  
    new Blue();  
}
```

D

A. cli
B. clicli

C. Does not compile
D. Does not run

Module 1 - Question 5

Which variables can be replaced by _?

```
void main(String[] args) {  
    enum Suit { HEART, DIAMOND, SPADE, CLUB };  
    enum Symbol { ACE, JACK, QUEENS, KING};  
    record Card(Suit suit, Symbol symbol) {}  
    var card = new Card(Suit.HEART, Symbol.ACE);  
    try {  
        if (card instanceof Card(Suit suit, Symbol symbol))  
            IO.println(suit);  
    } catch (NullPointerException e) {  
        IO.print("party!");  
    }  
}
```

A. args
B. card

C. symbol
D. e

Module 1 - Question 5

Which variables can be replaced by _?

```
void main(String[] args) {  
    enum Suit { HEART, DIAMOND, SPADE, CLUB };  
    enum Symbol { ACE, JACK, QUEEN, KING };  
    record Card(Suit suit, Symbol symbol);  
    var card = new Card(Suit.HEART, Symbol.ACE);  
    try {  
        if (card instanceof Card symbol symbol))  
            IO.println(suit);  
    } catch (NullPointerException e) {  
        IO.print("party!");  
    }  
}
```

C, D

A. args
B. card

C. symbol
D. e

Java 17-21 Features

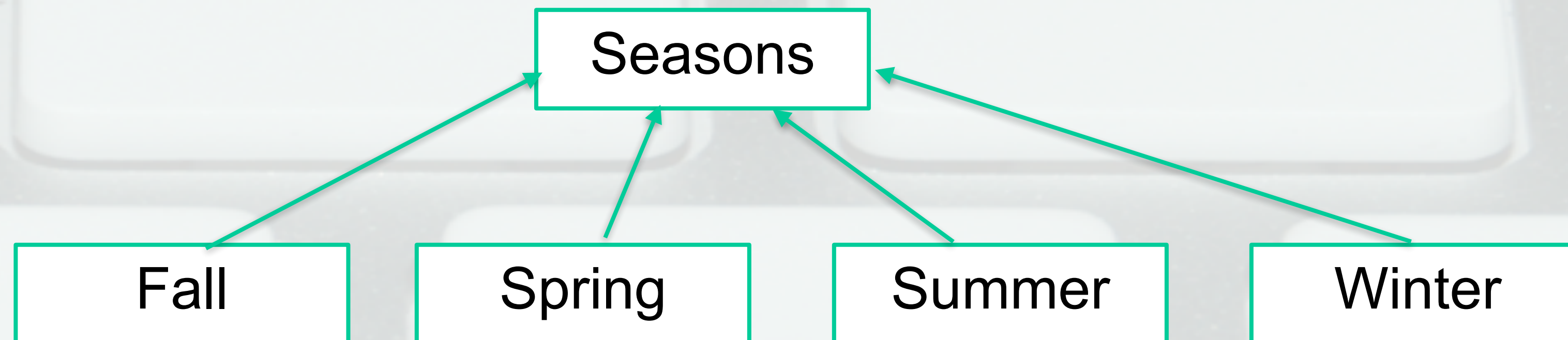
<https://linktr.ee/jeanneboyarsky>

Sealed classes

17

```
public abstract sealed class Seasons  
    permits Fall, Spring, Summer, Winter { }
```

```
final class Fall extends Seasons {}  
final class Spring extends Seasons {}  
final class Summer extends Seasons {}  
final class Winter extends Seasons {}
```



Subclass modifiers

17

Modifer	Meaning
final	Hierarchy ends here
non-sealed	Others can subclass
sealed	Another layer

Sealed interface

17

```
public sealed interface TimeOfDay
    permits Morning, Afternoon, Evening {
        boolean early();
    }
public non-sealed class Morning implements TimeOfDay {
    public boolean early() { return true; }
}
public non-sealed class Afternoon implements TimeOfDay {
    public boolean early() { return false; }
}
public record Evening(int hour) implements TimeOfDay {
    public boolean early() { return false; }
}
```

Records are implicitly final

Using Pattern Matching

21

```
return switch (obj) {  
  case null -> "";  
  case Integer i when i >= 21 -> "can gamble";  
  case Integer i -> "too young to gamble";  
  case String s when "poker".equals(s) -> "table game";  
  case String s when "craps".equals(s) -> "table game";  
  case String s -> "other game";  
  default -> throw new IllegalArgumentException("unexpected type");  
};
```

Annotations:

- null allowed (points to `case null`)
- guard clause (points to `when i >= 21`)
- pattern variable (points to `s` in `case String s`)

Still Null Pointer if don't specify case null

Order matters

21

```
return switch (obj) {  
    case null -> "";  
    case Integer i -> "too young to gamble";  
    case Integer i when i >= 21 -> "can gamble"; // DOES NOT COMPILE  
    case String s -> "other game";  
    default -> throw new IllegalArgumentException("unexpected type");  
};
```

Order like catching exceptions!

Enums

21

```
enum Suit {  
    HEART, DIAMOND, CLUB, SPADE;  
}
```

```
String symbol(Suit suit) {  
    return switch (suit) {  
        case HEART -> "♥";  
        case Suit.DIAMOND -> "♦";  
        case CLUB -> "♣";  
        case Suit.SPADE -> "♠";  
    };  
}
```

Fails compilation if miss one since no default

Record Patterns

21

```
enum Suit { HEART, DIAMOND, CLUB, SPADE; }  
enum Rank { NUM_2, NUM_3, NUM_4, NUM_5, NUM_6, NUM_7,  
  NUM_8, NUM_9, NUM_10, JACK, QUEEN, KING, ACE; }  
record Card(Suit suit, Rank rank) { }
```

```
if (card instanceof Card c) {  
    System.out.println(c.suit());  
    System.out.println(c.rank());  
}
```

Deconstructed

```
if (card instanceof Card(Suit suit, Rank rank)) {  
    System.out.println(suit);  
    System.out.println(rank);  
}
```


Nested Record Patterns

21

```
record MultiDeck(int deckNum, Card card) { }
```

```
if (multi instanceof MultiDeck(int deckNum,  
    Card(Suit suit, Rank rank))) {  
    System.out.println(deckNum);  
    System.out.println(suit);  
    System.out.println(rank);  
}
```


Switch pattern matching

21

```
switch(card) {  
  case Card(Suit suit, Rank rank)  
    when suit == Suit.HEART  
      -> System.out.println("Heart");  
  case Card c -> System.out.println("other");  
}
```


Switch pattern matching

21

```
String str = "";
switch (str) {
    case "heart" -> System.out.println("heart");
}
Card card = new Card(Suit.HEART, Rank.NUM_2);
switch (card) {
    case Card(Suit suit, Rank rank)
        when suit == Suit.HEART
        -> System.out.println("Heart");
    case Card c -> System.out.println("other");
}
```

Must be exhaustive if using “when”

What can't you do?

21

- In `switch(x)`, `x` still can't be
 - boolean
 - float
 - double
 - long
- Expanding non-records (coming ?)

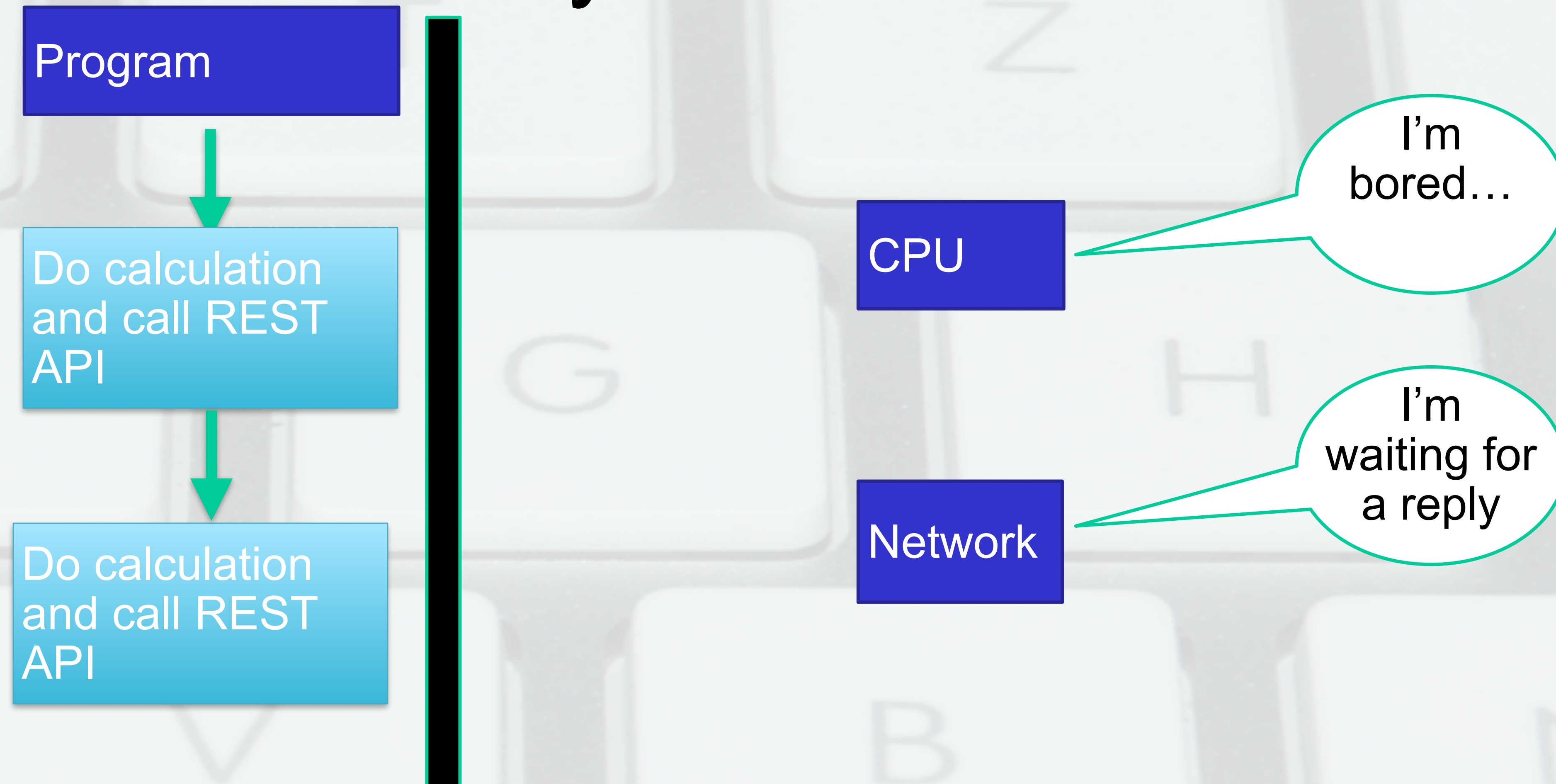
Leaps in Concurrency

21

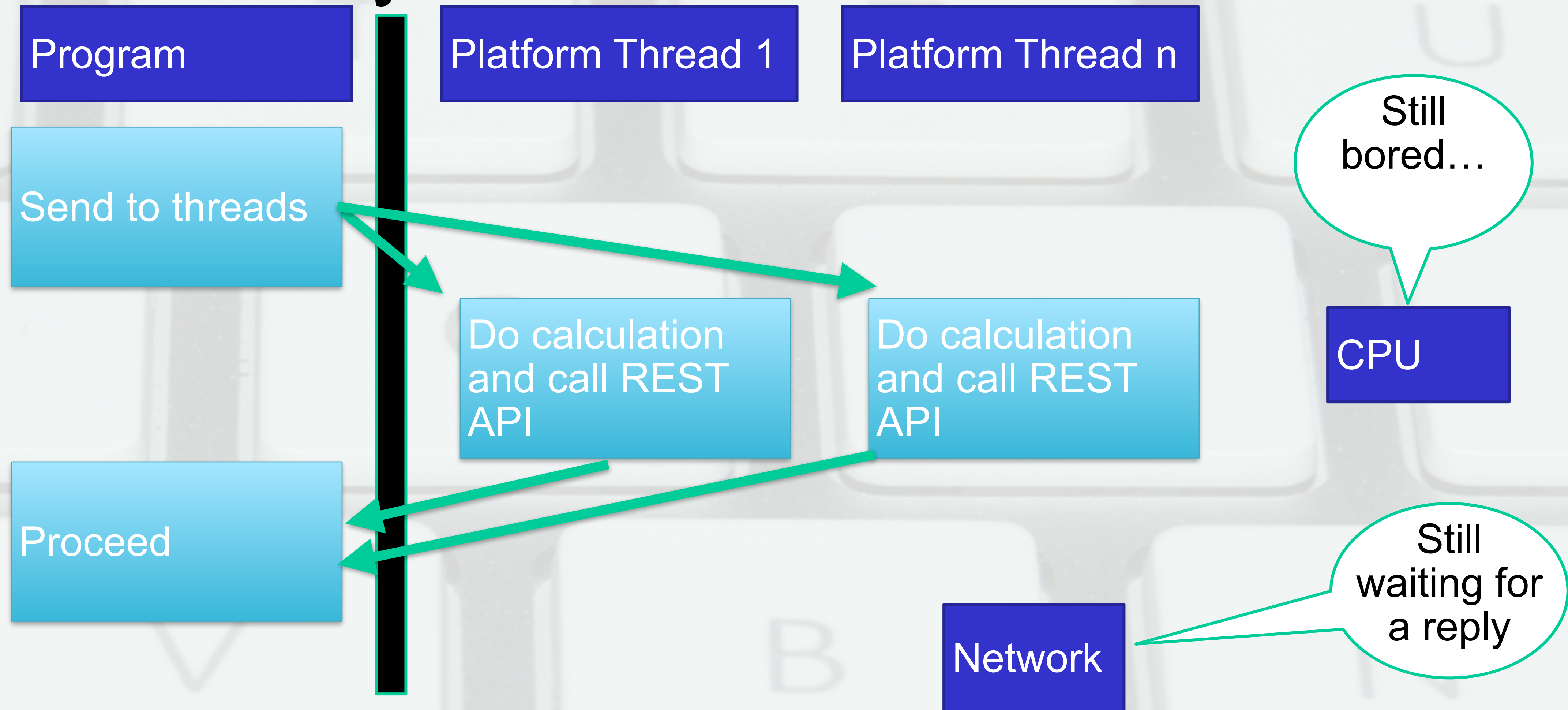


Plus many APIs...

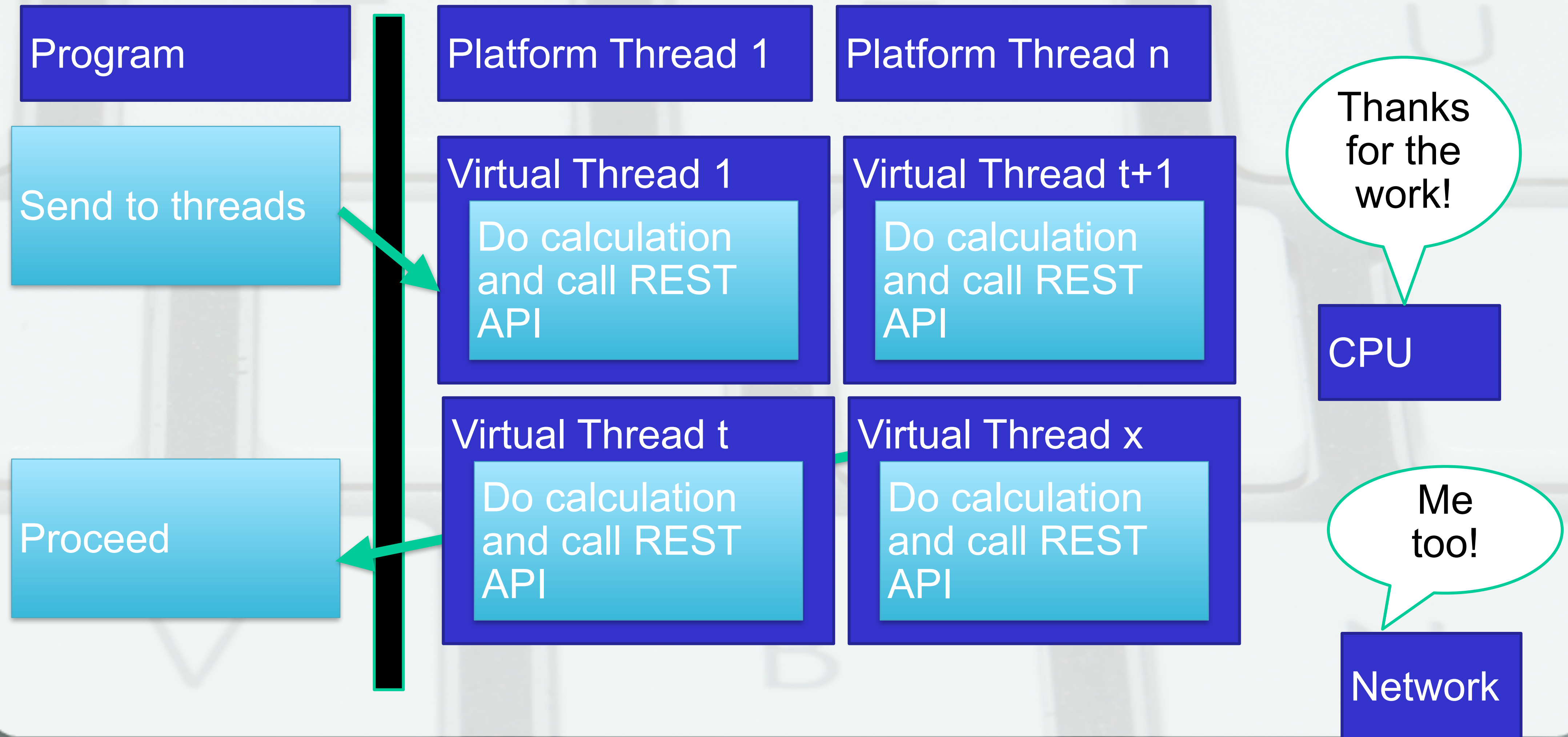
Why we need threads



Why we need virtual threads



With virtual threads



When most helpful?

21

- Thousands of threads
- Not CPU bound



Comparing

21

Platform (Traditional) Thread	Virtual Thread
Heavyweight	Lightweight
Tied to OS threads	Runs on OS thread but doesn't monopolize
Often pooled	Often shortlived
Instance of <code>java.lang.Thread</code>	Instance of <code>java.lang.Thread</code>

Fill in the blank

21

```
try (ExecutorService service = Executors._____) {  
    service.submit(() -> doStuff());  
}
```

Examples:

- `newSingleThreadExecutor()`
- `newFixedThreadPool(5)`
- `newCachedThreadPool()`
- `newVirtualThreadPerTaskExecutor()`

Autocloseable

19

Method	Java 21
shutdown()	No more tasks, but finish ones have
shutdownNow()	Stop tasks in progress
close()	Calls shutdown by default

If need directly

21

Platform Thread

Thread.ofPlatform()

new Thread(...)

Virtual Thread

Thread.ofVirtual()

Not via constructor

Module 2 - Question 1

How many compiler errors are in this code?

```
public sealed class Phone {  
  
    class iPhone extends Phone {  
    }  
  
    class Android extends Phone {  
    }  
}
```

A. 0
B. 1

C. 2
D. 3

Module 2 - Question 1

How many compiler errors are in this code?

```
public sealed class
```

```
    class iPhone {
```

```
    class Android {
```

```
}
```

C

(extending classes
needed to be sealed
or final)

A. 0

B. 1

C. 2

D. 3

Module 2 - Question 2

How many lines do you need to remove to make this code compile?

```
double odds(Object obj) {  
    return switch (obj) {  
        case String s -> throw new IllegalArgumentException("unknown game");  
        case String s when "blackjack".equals(s) -> .05;  
        case String s when "baccarat".equals(s) -> .12;  
        case Object o -> throw new IllegalArgumentException("unknown game");  
        default -> throw new IllegalArgumentException("known game");  
    };  
}
```

A. 0

B. 1

C. 2

D. 3

Module 2 - Question 2

How many lines do you need to remove to make this code compile?

```
double odds(Object obj) {  
    return switch (obj) {  
        case String s -> throw new IllegalArgumentException("unknown game");  
        case String s when "black" == s -> 5;  
        case String s when "black" == s -> 5;  
        case Object o -> throw new IllegalArgumentException("unknown game");  
        default -> throw new IllegalArgumentException("known game");  
    };  
}
```

C
String s & Object
o or default

A. 0

B. 1

C. 2

D. 3

Module 2 - Question 3

What is true about this class?

```
class Sword {  
    int length;  
  
    public boolean equals(Object o) {  
        if (o instanceof Sword sword) {  
            return length == sword.length;  
        }  
        return false;  
    }  
    // assume hashCode properly implemented  
}
```

- A. equals() is correct
- B. equals() is incorrect

C. equals() does not compile

Module 2 - Question 3

What is true about this class?

```
class Sword {  
    int length;  
  
    public boolean equals(Object o)  
        if (o instanceof Sword)  
            return length == o.length;  
        }  
    return false;  
}  
// assume hashCode properly implemented  
}
```

A

A. equals() is correct
B. equals() is incorrect

C. equals() does not compile

Module 2 - Question 4

What does `printLength(3)` print?

```
class Sword {  
    int length = 8;  
  
    public void printLength(Object x) {  
        if (x instanceof Integer length) {  
            length = 2;  
        }  
        System.out.println(length);  
    }  
}
```

A. 2

B. 3

C. 8

D. Does not compile

Module 2 - Question 4

What does `printLength(3)` print?

```
class Sword {  
    int length = 8;  
  
    public void printLength(  
        if (x instanceof Int  
            length = 2;  
        }  
        System.out.println(  
    }  
}
```

C

A. 2

B. 3

C. 8

D. Does not compile

Module 2 - Question 5

21

What is true doStuff() at line v1?

```
try (var service = Executors.newVirtualThreadPerTaskExecutor()) {  
    service.submit(() -> doStuff());  
}
```

// line v1

- A. doStuff() may be running
- B. doStuff() was killed
- C. doStuff() completed
- D. None of the above

Module 2 - Question 5

21

What is true doStuff() at line v1?

```
try (var service = Executors.newVirtualThreadPerTaskExecutor()) {  
    service.submit(()  
}  
// line v1
```

C

- A. doStuff() may be running
- B. doStuff() was killed
- C. doStuff() completed
- D. None of the above

Module 2 - Question 6

I learned a lot today and I'm ready to code

A. True

B. False

Lab:<https://github.com/boyarsky/2026-java-one>

Note that longer than will have time for. Can finish on own or during my Wed 3pm hack session